IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

Benchmarking Deep Reinforcement Learning for Cartesian Path Tracking of Free-Floating Space Robot Manipulators

PATRICK S. ERMISCH[1], and ERIC GUIFFO KAIGOM[1]
[1]Department of Computer Science & Engineering, Frankfurt University of Applied Sciences, Frankfurt am Main, Germany
(e-mail: steven.ermisch@stud.fra-uas.de, kaigom@fra-uas.de)

Corresponding author: Eric Guiffo Kaigom (e-mail: kaigom@fra-uas.de).

ABSTRACT Free-floating space robots exhibit strong dynamic coupling between manipulator motion and base attitude disturbance, complicating Cartesian path tracking of the end-effector. Adopting data-driven approaches is challenging due to limited data availability in the space sector. This paper explores trial-and-error learning techniques for Cartesian path tracking in a physics-based Simulink/Simscape environment with integrated safety monitoring. Since quantitative and qualitative comparison of these methods is currently missing in on-orbit servicing, we benchmark six Reinforcement Learning agents: TRPO, TD3, SAC, PPO, Policy Gradient, and DDPG. We evaluate their performance using unified key performance indexes including training duration and stability, tracking accuracy, base attitude disturbance, motion smoothness, energy consumption, and collision risks. PPO achieves the best overall trade-off between tracking accuracy, base disturbance minimization, and training stability. Building on this, we quantify the effect of systematic tuning of neural network architecture and key hyperparameters of PPO for tracking both circular and ramp-shaped Cartesian paths. Compared to default PPO configuration, the optimized version improves mean episodic return by 80.7%, reduces mean squared tracking error by 67.7% and maximum tracking error by 44.1%, and stabilizes base orientation (67.0% reduction). Moreover, optimized PPO eliminates collisions and joint-limit violations while reducing energy consumption by 15.98%.

INDEX TERMS free-floating manipulation, Proximal Policy Optimization (PPO), reinforcement learning, safe simulation, space robotics, trajectory tracking

I. INTRODUCTION AND BACKGROUND

Free-floating robot manipulators operating in the space environment to inspect, capture, or de-orbit satellites introduce unique control challenges when compared to terrestrial robots [1]. In micro-gravity, the robot's base (typically a spacecraft) is not fixed and thrusters are turned off. Manipulator motions can give rise to a change of the position and orientation of the base, which in turn modifies the final pose of the end-effector. This reaction is due to the conservation of the linear and angular momentum [2]. The free-floating dynamics of space robots has many benefits. An expensive consumption of fuel, such as hydrazine (N_2H_4), and excitation of the robot dynamics for some activation frequencies of the pulse width modulation related to thruster control can be avoided [3].

However, a free-floating base complicates the achievement of robotized manipulations. Indeed, the end-effector of the arm may fail to reach the desired target point if standard motion planning approaches dedicated to robots with a fixed base are simply applied. Further constraints, such as communication latency and limited onboard computational resources, make more intricate the continuous real-time control and stabilization of the base [1], [4]. In fact, traditional control approaches that assume an accurate model of the robot dynamics (e.g. reactionless planning or precise attitude control) can struggle to adapt to uncertainties and unmodeled dynamics. Data-driven approaches, on the other hand, hardly apply. Usually, the required amount and quality of data are rather unavailable in the space sector. In many cases, robotized tracking capabilities under a free-floating behavior of the arm's base need to be demonstrated long before the physical robot system is assembled and deployed [5]–[7]. These challenges motivate the development and simulation of AI-empowered digital twins under desired operational conditions to explore learning-based control methods that can autonomously adapt to complex and unknown dynamics even

in the case of data scarcity.

Reinforcement Learning (RL) has emerged as a promising approach for robotics, enabling agents to learn control policies through trial-and-error interactions with entities, environments, and processes. Recent advances in deep RL have shown success in complex control tasks, but applying RL to space robots is still relatively novel [8], [11]–[13], [15].

The work presented in this paper investigates the use of continuous-action deep RL for the tracking of a Cartesian motion of the end-effector of a free-floating space manipulator. It differs from previous contributions in three ways. First, it focuses on a systematic comparison and explores the respective performance of six different RL algorithms beyond those mentioned e.g. in Table 1, thereby quantitatively and qualitatively contributing to filling in a current gap in space robotics. Second, several constraints, such as collision avoidance, are monitored through formal methods applied from a software-enabled and practical perspective. Finally, this work investigates the minimization of the base attitude in conjunction with constraint monitoring previously mentioned. Recall that the induced base disturbance is omitted in most recent contributions, as highlighted in Table 1. We are not aware of any comprehensive work that compares RL algorithms for the Cartesian path tracking of free-floating space robots while considering the base attitude minimization and unifying constraint monitoring by leveraging formal methods, to the best of our knowledge (see, also Table 1).

The key contributions and novelties of this work include:

- A comparative evaluation of RL Agents: We implement and evaluate six state-of-the-art continuous RL algorithms, Trust Region Policy Optimization (TRPO) [16], Twin Delayed Deep Deterministic Policy Gradient (TD3) [17], Soft Actor-Critic (SAC) [18], Proximal Policy Optimization (PPO) [19], vanilla Policy Gradient (PG) [20], and Deep Deterministic Policy Gradient (DDPG) [21], on the same free-floating robot control task. This side-by-side comparison under unified conditions reveals which algorithm is best suited for controlling a free-floating space robot and why.
- An integration of formal safety measures: To meet safety-critical space operation requirements, we incorporate formal methods into the learning loop [2], [22]–[24]. A collision monitoring, momentum conservation check, and formal verification of joint torque limits are embedded in the simulation. This integration ensures the RL agent cannot violate fundamental safety properties (like avoiding collisions or exceeding actuator limits) during training or execution.
- A high-fidelity simulation environment: We develop a high-fidelity MATLAB/Simulink simulation of a free-floating space robot, using a URDF-based multibody model and the Simscape physics engine. The robot, a cube-shaped base with a 4-Degree of Freedom (DOF) robotic arm, is modeled with realistic kinematics, dynamics, and sensor/actuator parameters and features. A dedicated reinforcement learning environment provides the agent with observations and rewards, including a specially designed reward function that incentivizes accurate end-effector trajectory tracking while penalizing base movement, large joint velocities, and excessive control effort (energy). Through reward shaping and carefully chosen reset conditions, the agent learns to accomplish the task without causing excessive base disturbance or safety violations [21], [25]–[28].
- The optimization of the PPO agent : Based on the comparative results, we identify PPO as the top- performing algorithm and further refine it for this task. We adjust the neural network architecture and tune hyperparameters (e.g., learning rate, batch size, clipping ratio, entropy regularization, etc.) to improve training stability and policy performance. The optimized PPO agent achieves significantly better tracking accuracy, smoother control inputs, and zero safety violations, outperforming the default configuration on all key metrics.

Overall, our study demonstrates that a combination of multiple-body simulation, deep RL, and formal verification can yield a robust and safe control policy for a free-floating space robot. In the following, we first describe the system modeling and RL framework, then detail the formal safety components, present a comparative analysis of the RL agents, and finally discuss the optimized PPO agent's performance, before concluding with broader insights and future directions.

II. RELATED WORK

A. REINFORCEMENT LEARNING FOR ROBOTIC CONTROL

RL has demonstrated success in various robotic manipulation tasks [21], [29]. Policy gradient methods (PG, PPO, TRPO) and actor-critic algorithms (DDPG, TD3, SAC) have been widely applied to continuous control problems [16]–[20].

B. CONTROL OF FREE-FLOATING SPACE ROBOTS

Traditional approaches for space robot control include computed torque control, reactionless path planning, and attitude regulation [1], [2]. Recent works have begun exploring RL for path planning [11], [15], obstacle avoidance [8], and trajectory tracking [12].

C. SAFE REINFORCEMENT LEARNING

Safe RL integrates safety constraints through formal verification, barrier functions, and constrained optimization [23], [24], [30], [31]. However, application to space robotics with formal safety guarantees remains limited.

D. RESEARCH GAP

Despite recent advances, systematic comparisons of RL algorithms for free-floating space robot control are lacking. Most prior works focus on single algorithms and omit base attitude minimization or formal safety verification (see Table 1).

TABLE 1: Recent advances in Deep RL-based motion management for free-floating space robots.

Contribution	RL Algorithm	Task Objective	Formal Verification/ Base Disturbance Min.
[8], 2023	SAC	Obstacle avoidance	No/No
[9], 2024	DDPG	Collision avoidance	No/No
[10], 2024	PPO	Adaptation to joint failure	No/No
[11], 2024	DDPG	Path planning	No/No
[12], 2025	PPO	Trajectory planning	No/No
[13], 2025	LSAC	Trajectory post-processing	No/No
[14], 2025	DDPG	Trajectory planning	No/Yes

III. SYSTEM MODELING AND SIMULATION ENVIRONMENT

A. ADVANTAGES OF AI-ENDOWED DIGITAL TWIN TECHNOLOGY IN ON-ORBIT SERVICING

Recent advances in AI-driven digital twinning have demonstrated considerable benefits for on-orbit servicing, ranging from automated lifecycle management of satellites to the orchestration of skillful agents that autonomously carry out complex assembly, integration, and testing tasks [32], [33]. By combining contextualized real-time sensor data with AI that accommodates uncertainties, digital twins bridge the physical and digital world with accelerated simulation-driven analysis at low costs. Verification and validation also benefit from this paradigm, as changes in system design or operational scenarios can be described in scenario spaces and integrated with digital twins for flexible verification [34].

B. ROBOT MODEL

The virtual testbed for this research allows for the simulation of a free-floating space manipulator inspired by the Spacecraft Robotics Toolkit (SPART) example [35]. The robot consists of a cuboid base (serving as the spacecraft) with a 4-degree-of-freedom (4-DOF) serial manipulator arm mounted on it (see Figure 1). The arm has four revolute joints and four links, and its kinematic structure and inertial properties are defined in a Unified Robot Description Format (URDF) file (SpaceRobot.urdf) which is imported into Simulink's multibody simulation environment [25]–[27]. Identified values (see, .e.g., [36]) were chosen for the geometry and the inertial properties of links of the space arm considered in this work and stored in the URDF model. The result is a realistic multibody model with a floating base and an arm, capturing the strong dynamic coupling between them [2].

C. DYNAMIC SIMULATION IN MATLAB/SIMULINK

The simulation is implemented in Simulink using the Simscape Multibody physics engine. Gravity is turned off (set to [0 0 0]) to emulate the microgravity orbital environment. The base is connected to the inertial world frame through an unactuated 6- DoF joint [37], allowing it to move freely in translation and rotation. No external forces or torques act on the system. Thus, the base only moves in response to forces generated by the manipulator's joints. This configuration ensures that momentum is conserved, motions of the arm can lead to a reaction of the base [2], [28], [31], [37].

The Simulink model is organized into modular subsystems, as illustrated in Figure 2. The Robot Plant subsystem computes the full 6-DOF multibody dynamics of the free-floating robot at each time step (see Figure 3), taking joint torque inputs and outputting the resulting states by numerically integrating the equations of motion [2]. The Controller subsystem executes the learned RL policy, applying joint torque commands that pass through saturation and safety verification blocks before reaching the plant. If the collision monitor signals an imminent collision, this block issues an emergency stop and gradually overrides all torques to zero [23], [24]. The Reference Trajectory Generator provides the desired end-effector trajectory and base attitude (see Figure 4), outputting the current target position and velocity at each time step. Various sensors measure the robot's state (joint angles, velocities, base orientation, base angular velocity), combining them into an observation signal for the RL agent (detailed in section IV). Finally, safety monitors enforce constraints: a Collision Monitor checks distances between robot components and triggers a safety halt if any distance falls below a threshold d_{safe}, while a joint-limit monitor triggers an emergency stop if any joint exceeds its allowed range [23], [24], [38].

To accurately simulate the continuous dynamics while interfacing with the discrete-time RL agent, a consistent integration scheme is used. The Simscape solver is configured to operate with a fixed-time step equal to the agent's decision interval (Δt), instead of the default variable-step, to ensure synchronization between the physics simulation and the RL decision-making cycle. Rate transition blocks handle any rate differences, avoiding aliasing and ensuring that state observations and action commands align appropriately between the continuous plant and the discrete agent loop [39], [40]. Additionally, Simulink's unit consistency checks are enabled to catch any modeling errors (e.g. mismatched units) early in development.

D. REFERENCE TRAJECTORY AND DESIRED TASK

The target task for the robot is to follow a periodic end-effector trajectory while minimizing base disturbance. As previously mentioned, the reference trajectory is chosen as a circular path (see Figure 1). Prior to training the RL agents, the trajectory was validated via inverse kinematics to ensure

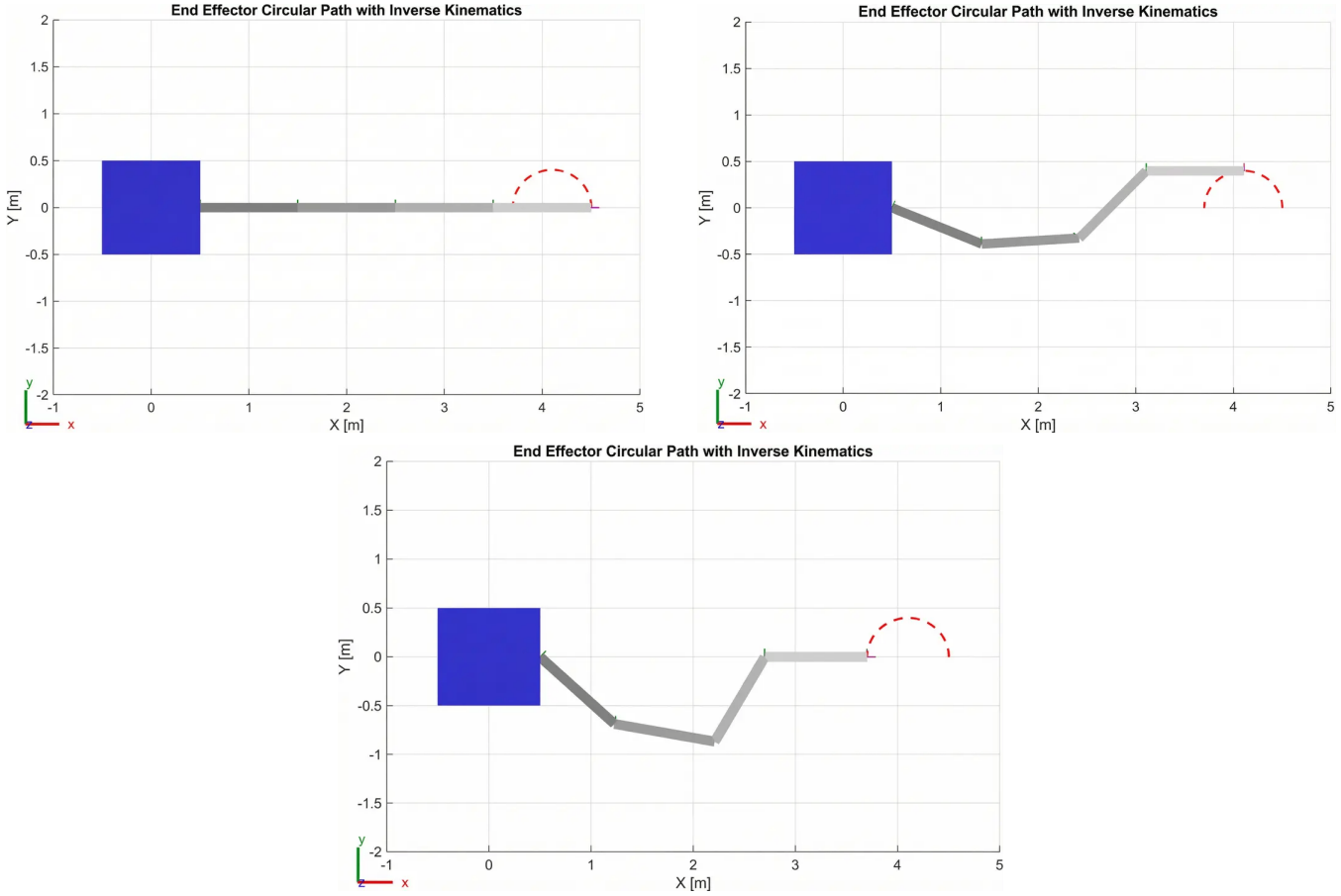


FIGURE 1: Tracking the desired end-effector circular path using inverse kinematics.

it is kinematically feasible for the manipulator (within joint limits and avoiding singularities). The inverse kinematics solution produced a consistent set of joint movements that achieve the desired end-effector motion, confirming that the circle can be tracked by the arm without requiring unrealistic motions from a kinematic viewpoint. This precaution guarantees that the task is achievable, at least by some controller, so if the RL agent fails to learn it, it is due to learning limitations rather than an unattainable desired Cartesian poses of the end-effector.

During simulation, the reference subsystem outputs the desired end-effector position and velocity as time series $EE_{ref}(t)$ and $EE_{vref}(t)$ in the workspace at each time step. From these terms, the instantaneous tracking error signals are computed for the position and velocity of the end-effector. These error signals are key components of the observation and are also used in the reward computation that penalizes large errors.

IV. REINFORCEMENT LEARNING FRAMEWORK

State (Observation) Space: The RL agent receives an observation capturing the system's key state information relevant to control at each time step [21]. The state vector is composed of the following elements:

- End-Effector Tracking Error: The position error $e_p \in \mathbb{R}^3$ between the end-effector's current position and the reference target position (in Cartesian coordinates). If the end-effector is exactly on the desired trajectory point, $e_p = 0$. Additionally, the error in end-effector velocity $e_v \in \mathbb{R}^3$ (difference between current and target end-effector velocities) is included. Together, (e_p, e_v) provides the agent a direct indication of tracking performance. Specifically, it tells the agent how far off the trajectory the end-effector is and whether it is lagging or overshooting in velocity.
- Arm Joint States: The positions and velocities of each of the 4 arm joints. We include the joint angles $q \in \mathbb{R}^4$ and joint angular velocities $\dot{q} \in \mathbb{R}^4$ in the observation. These inform the agent about the manipulator's configuration and motion state (e.g., at rest or not). (Note: Joint angles can be normalized or wrapped as needed, in our implementation they are provided in radians along with known joint limits.)
- Base Motion: The free-floating base's orientation and angular velocity. We parameterize the base's orientation using quaternions. The quaternion $q_{base} \in \mathbb{R}^4$ describes the orientation of the base, capturing its attitude error from a reference orientation. Since the base is expected

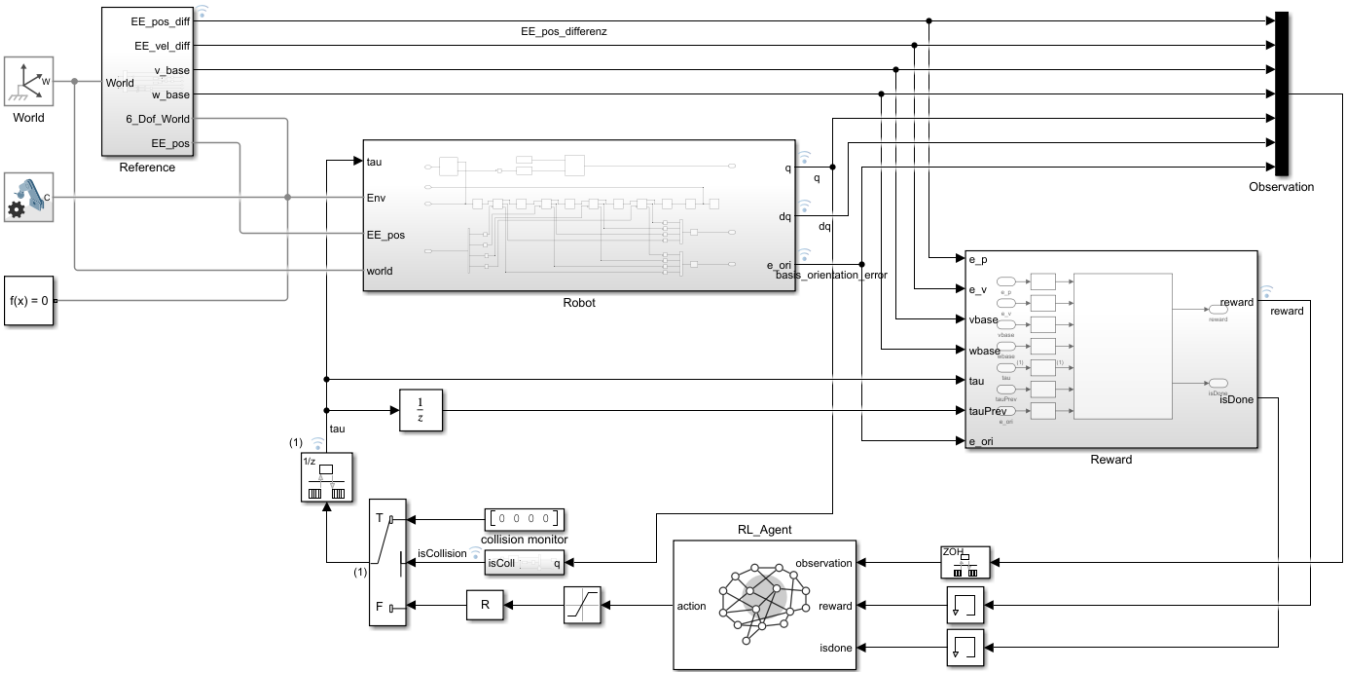


FIGURE 2: Overview of the Simulink model `SpaceRobot.slx`
This figure shows the Simulink-based closed-loop simulation framework for the free-floating space robot. The model integrates the robot dynamics, reference generation, observation assembly, reward computation, and a PPO-based reinforcement learning agent that outputs joint torques. The reward and termination logic evaluate end-effector tracking performance, base motion disturbance, and control effort to guide policy learning.

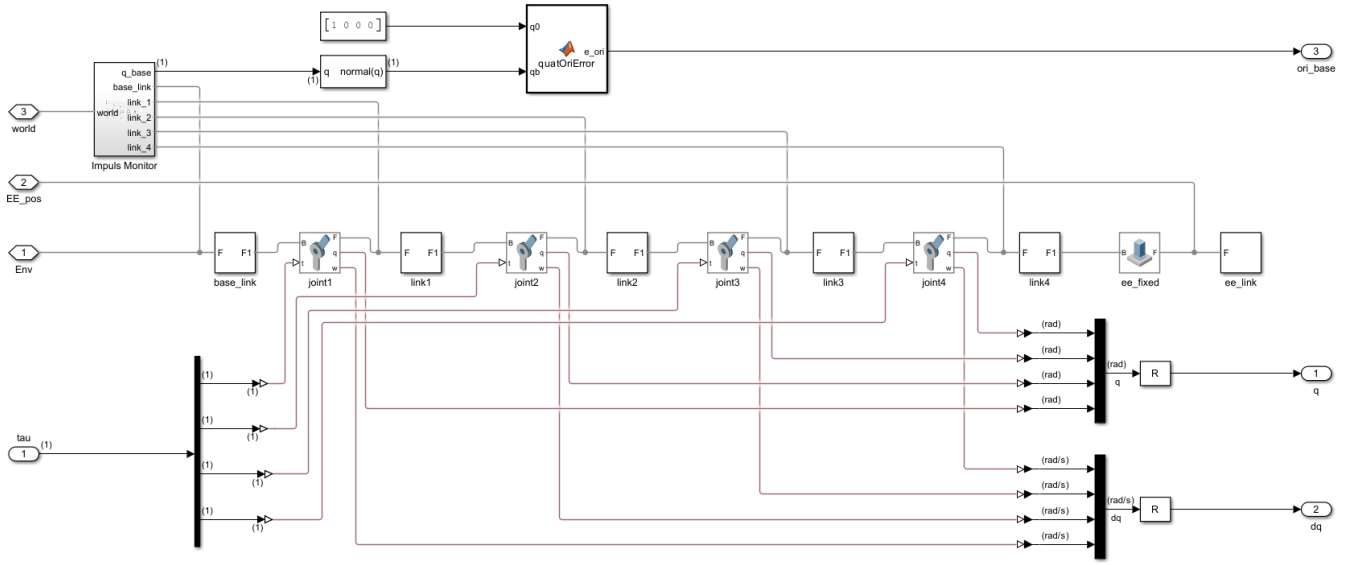


FIGURE 3: Overview of the Robot Subsystem in the Simulink Model `SpaceRobot.slx`
This figure depicts the Simulink/Simscape Multibody model of the free-floating space robot, consisting of a floating base and a serial 4-DOF manipulator. Joint torque commands are applied to the revolute joints, while joint positions and velocities are measured and fed back as state variables. The base orientation is represented as a normalized quaternion and compared to a reference to compute the base attitude error.

to maintain its initial attitude, the orientation error is ideally kept near zero through the agent's actions. The base's angular velocity $\omega_{base} \in \mathbb{R}^3$ is also included, as it captures any ongoing rotation of the base.

Combining these components, the overall observation vector has dimension $3(e_p) + 3(e_v) + 4(q) + 4(\dot{q}) + 3(ori_{base}) +$

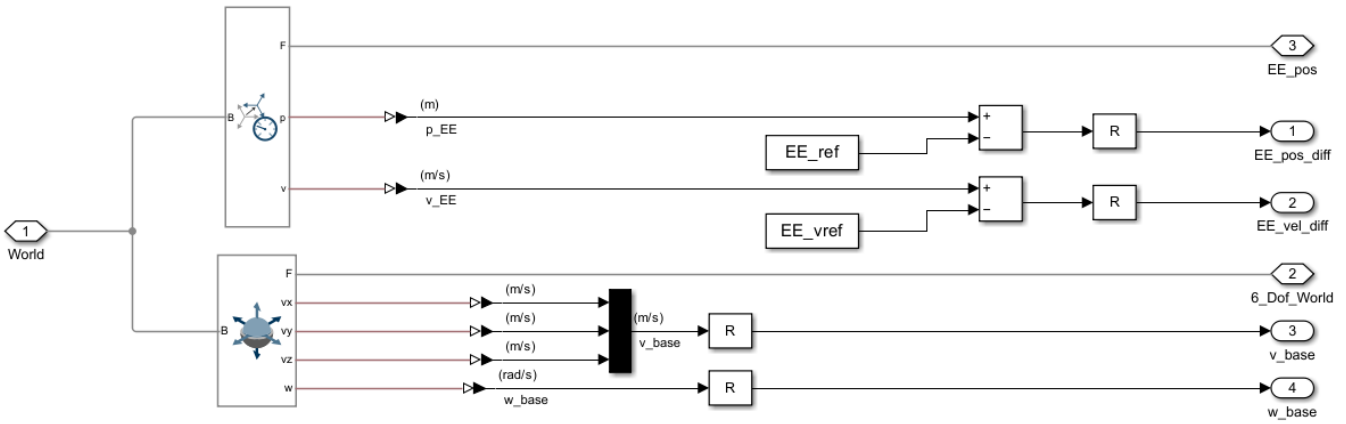


FIGURE 4: Overview of the Reference Subsystem in the Simulink Model `SpaceRobot.slx`
This figure shows the computation of end-effector position and velocity tracking errors with respect to the reference trajectory. The measured end-effector states are compared to the reference signals to generate position and velocity error vectors used for observation and reward calculation. In addition, the base linear and angular velocities are extracted to capture base motion induced by manipulator actuation.

$3(\omega_{base}) + 3(v_{base}) = 23$ dimensions. This rich state description provides the agent with all necessary information: how far the end-effector is from target, how the arm is configured/moving, and how the base is reacting.

Action Space: The agent's action is a vector of continuous joint torque commands for the manipulator's joints. At each decision step, the agent outputs $a = [\tau_1\ \tau_2\ \tau_3\ \tau_4]^T$, corresponding to the torque to apply at each of the 4 revolute joints of the arm [21], [29]. These actions are real-valued and are constrained by the actuator capabilities. We enforce symmetric torque limits: each $|\tau_i| \leq \tau_{\max}$ (for i = 1..4). In the simulation, this is implemented using saturation blocks that clip the commanded torque to the range $[-\tau_{\max}, +\tau_{\max}]$. The value of $\tau_{\max}$ is set based on the robot's design (for example, $\tau_{\max} = 25Nm$ for all joints, though the exact value is not critical as long as it is sufficient to follow the trajectory). These saturation limits not only model physical actuator limits but also serve as a safety measure to prevent excessive torques. The action space is thus continuous $\mathbb{R}^4$, making this a continuous control problem that suits policy gradient and actor-critic RL methods.

The agent's policy (the controller we are learning) is a mapping from the 23-dimensional state space to the 4-dimensional action space: $\pi : s \mapsto a$. Internally, this policy is represented by a neural network whose parameters are adjusted during training to maximize cumulative reward. The control frequency (agent decision frequency) is chosen such that the agent outputs a new torque command at a fixed interval Δt (e.g., 0.1 s). This Δt is the "agent sampling time" in Simulink and is consistent with the simulation's fixed step to maintain alignment.

Reward Function Design: A crucial aspect of the reinforcement learning formulation is the reward signal, which guides the agent toward accurate trajectory tracking while minimizing base disturbance and enforcing smooth, safe control actions. At each discrete time step t, the reward r_t is computed as the sum of a progress term and a proximity bonus, minus a weighted cost function:

$$r_t = \frac{1}{C}\left(r_{\text{prog}} + r_{\text{bonus}} - c_t\right), \tag{1}$$

where C is a normalization constant.

To encourage monotonic convergence toward the reference trajectory, a progress reward is defined based on the reduction of the end-effector position error:

$$r_{\text{prog}} = \begin{cases} k_p\left(d_{t-1} - d_t\right), & d_{t-1} > d_t, \\ 0, & \text{otherwise}, \end{cases} \tag{2}$$

where $d_t = \|e_p(t)\|$ is the Euclidean end-effector position error and k_p is a scaling factor. The previous distance d_{t-1} is stored persistently across time steps.

A small shaping bonus is added when the end-effector is close to the reference point:

$$r_{\text{bonus}} = k_b \exp\left(-\left(\frac{d_t}{\sigma}\right)^2\right), \tag{3}$$

which improves local convergence behavior near the desired trajectory.

The instantaneous cost c_t penalizes tracking errors, base motion, and control effort:

$$\begin{aligned} c_t = {} & W_p\|e_p\|^2 + W_v\|e_v\|^2 + W_{\text{ori}}\|e_{\text{ori}}\|^2 + W_{\omega_b}\|\omega_{\text{base}}\|^2 \\ & + W_{v_b}\|v_{\text{base}}\|^2 + W_u\|\tau\|^2 + W_d\|\tau - \tau_{t-1}\|^2, \end{aligned} \tag{4}$$

where e_p and e_v denote the end-effector position and velocity errors, e_{ori} the base orientation error, v_{base} and ω_{base} the linear and angular base velocities, and τ the joint torque vector. The final term penalizes changes in control input to promote smooth actuation.

The weights used in all experiments are summarized in Table 2. High weights are assigned to end-effector tracking and

base orientation preservation, while control effort and torque-rate penalties are kept comparatively small to allow sufficient exploration.

At each step, all inputs to the reward function are checked for numerical validity. If any NaN or Inf values are detected, or if the end-effector error exceeds a predefined bound, the episode is immediately terminated and a terminal reward of -1 is assigned. This mechanism prevents the agent from exploiting invalid states and ensures numerical robustness during training.

All reward components were implemented in a MATLAB Function block within the Simulink environment. The weights were tuned empirically to provide informative gradients while maintaining stable learning behavior.

Training Procedure: We adopt an episodic training approach. Each episode corresponds to a simulation of fixed duration. A finite horizon of H seconds or a fixed number of time steps N. In our case, one episode lasts long enough for the end-effector to attempt following the circular trajectory (e.g., one or two loops of the circle). At the start of each episode, the system is reset to an initial condition by a custom reset function. The initial joint angles are set to a feasible configuration (e.g., the arm in a certain pose near the start of the trajectory) and the base is at a defined attitude (often the home orientation and at rest). All state variables, observer integrators, and logging signals are reset. We ensure the initial state is within safe bounds (no collisions or limit violations at t=0). The reset function can randomize the starting joint angles within a range or the phase of the trajectory to improve generalization, though within this study, we kept a consistent start pose for simplicity (with the end-effector at a specific point on the circle) [21].

Once an episode begins, the agent receives observations at each step and outputs torque actions. The simulation runs until either the time horizon is reached (the episode length H) or a terminal condition occurs (task completed or safety violation). As described, reaching the end of the trajectory successfully or experiencing a failure condition will terminate the episode prematurely. In practice, an episode might end early if, for example, the agent causes a collision at 70% of the trajectory, it will receive the penalty and the episode stops there.

We use a standard policy gradient training loop (as provided by MATLAB's RL toolbox): after each episode (or after a set number of steps), the collected state, action, reward trajectories are used to update the agent's policy (and value function, if applicable) via the chosen RL algorithm [41]. The particularities of each algorithm (PPO, DDPG, etc.) are handled by their respective implementation, we supply the experience and let the algorithm update the neural network [21]. All agents were trained for the same number of episodes to allow fair comparison. We chose $N_{train} = 1000$ episodes for each agent, which was sufficient for most algorithms to converge or reach a performance plateau. During training, we monitored metrics such as the cumulative reward per episode, the end-effector tracking error, and the base motion, averaged over each episode. A successful training run is characterized by the episode reward increasing (toward 0, since negative costs are used) and the tracking error and base movement decreasing over time. We also tracked the number of episodes that ended prematurely due to safety triggers, aiming for this to go to zero as training progresses.

For each algorithm, we performed multiple training runs and selected the best-performing policy (based on highest average reward achieved) for final evaluation. We found relatively consistent results across runs for the more stable algorithms (PPO, TRPO), whereas some runs of less stable methods (e.g. DDPG) would diverge, underscoring the importance of stability.

Evaluation and Metrics: After training, we evaluated each agent's performance on $N_{eval} = 30$ independent episodes (with various initial conditions) to gather statistics. Over these evaluation runs, we compute a set of Key Performance Indicators (KPIs) to quantitatively compare the agents. These KPIs include:

- K1: Average Episodic Return, the mean cumulative reward per episode (higher is better, this reflects overall success in the task).
  $R_i = \sum_{k=1}^{T_i} r_i[k]$,
  $K1 = \frac{1}{N_{eval}} \sum_{i=1}^{N_{eval}} R_i$.
- K2: Mean Squared End-Effector Position Error, averaged over the episode (lower is better, measures tracking accuracy).
  $K2 = \frac{1}{N_{eval}} \sum_{i=1}^{N_{eval}} \frac{1}{T_i} \sum_{k=1}^{T_i} \|e_{p,i}[k]\|_2^2$.
- K3: Maximum End-Effector Position Error, worst-case deviation during the episode (lower is better).
  $K3 = \max_i \max_k \|e_{p,i}[k]\|_2$.
- K4: Mean Base Orientation Error, e.g., average quaternion error magnitude indicating how far the base drifted in attitude (lower is better).
  $K4 = \frac{1}{N_{eval}} \sum_{i=1}^{N_{eval}} \frac{1}{T_i} \sum_{k=1}^{T_i} \|e_{R,i}[k]\|_2$.
- K5: Collision Rate, the fraction of episodes in which a collision occurred (0 means no collisions, we expect this to be zero if the agent behaves safely).
  $C_i = \mathbb{I}(\exists k : c_i[k] > 0.5)$,
  $K5 = \frac{1}{N_{eval}} \sum_{i=1}^{N_{eval}} C_i$.
- K6: Joint Limit Violation Rate (0 is ideal).
  $K6 = \frac{1}{N_{eval}} \sum_{i=1}^{N_{eval}} \frac{1}{T_i n_J} \sum_{k=1}^{T_i} \sum_{j=1}^{n_J} \mathbb{I}(q_{i,j}[k] \notin$

TABLE 2: Reward function weights used in all experiments.

Term	Symbol	Value
End-effector position error	W_p	150
End-effector velocity error	W_v	25
Base orientation error	W_{ori}	200
Base angular velocity	W_{ω_b}	8
Base linear velocity	W_{v_b}	2
Joint torque magnitude	W_u	0.02
Torque rate (smoothness)	W_d	0.06
Progress scaling factor	k_p	10
Proximity bonus scaling	k_b	0.05
Proximity width	σ	0.02
Reward normalization constant	C	500

$[q_j^{\min}, q_j^{\max}]\big)$.

- K7: Control Smoothness (Jerk), a measure of actuation smoothness, computed as the second difference of joint torques. Lower values indicate smoother control.
  $\Delta^2\tau_i[k] = \tau_i[k+2] - 2\tau_i[k+1] + \tau_i[k]$,
  $K7 = \frac{1}{N_{eval}} \sum_{i=1}^{N_{eval}} \frac{1}{T_i - 2} \sum_{k=1}^{T_i-2} \|\Delta^2\tau_i[k]\|_2$.
- K8: Mean Base Angular Velocity, a measure of residual base rotation (rad/s) during the task (lower is better, ideally zero).
  $K8 = \frac{1}{N_{eval}} \sum_{i=1}^{N_{eval}} \frac{1}{T_i} \sum_{k=1}^{T_i} \|\omega_{b,i}[k]\|_2$.
- K9: Energy Consumption, approximate average energy expended by the joints (e.g., $\int(\tau \cdot \dot{q})dt$ using an absolute-power proxy). Lower energy usage is better if achieved without sacrificing performance.
  $E_i = \int \sum_{j=1}^{n_J} \left|\tau_{i,j}(t)\dot{q}_{i,j}(t)\right| dt$,
  $K9 = \frac{1}{N_{eval}} \sum_{i=1}^{N_{eval}} E_i$.

Additionally, we consider training-related metrics for comparison: T1: variance of the episodic reward in the later stage of training (indicating training stability), T2: variance of the value function or average reward (another stability metric), and T3: training time required to reach 1000 episodes (reflecting sample efficiency and computational load). These help evaluate how easily an algorithm learns in this environment.

By evaluating all agents on these common metrics, we obtain an objective comparison of their performance and characteristics. Section VI presents and discusses these comparative results.

V. SAFETY MONITORING AND CONSTRAINT ENFORCEMENT

Stability and safety are paramount in space robotics. A controller for autonomous space servicing must meet physical constraints and avoid hazardous behaviors, especially when it builds on learned skills expected to handle uncertainties. In this work, we integrate runtime safety monitors and constraint enforcement mechanisms directly into the RL training and execution environment. These mechanisms serve a dual purpose: they protect the simulation from unsafe states during exploration and, through penalty-driven learning, shape the agent's behavior toward safe operation [21], [23].

Momentum Conservation Monitor: In a free-floating system without external forces, the total linear momentum of the robot (base + arm) should remain constant (and in our case, near zero since initial momentum is zero) [2]. We implemented a Momentum Observer that continuously computes the total linear momentum p_{tot} of the system and checks for its conservation (see Figure 5). This is done by attaching virtual sensors to each body (base and links) to measure their velocities and computing $p_{tot} = \sum_i m_i v_i$. In an ideal simulation, p_{tot} stays roughly zero at all times (small numerical deviations on the order of 10^{-6} Ns may occur due to integration errors). If a significant, systematic change in p_{tot} were detected, it would indicate an unintended external force or a modeling error (such as an unmodeled constraint or bug). During our experiments, p_{tot} remained near 0 (within 10^{-6} Ns) throughout each episode with the learned policy, confirming physical consistency (see Figure 6). This momentum check provides confidence that the simulation adheres to Newton's laws and the agent interacts with a physical correct process. This leverages the significance of the obtained policy. The Momentum Conservation Monitor also serves as a diagnostic tool because any momentum drift would pause training for model inspection.

Collision Monitor and Avoidance: We define a minimum allowable distance between certain parts of the robot (for example, between the end-effector and the base structure). A Collision Monitor subsystem computes distances between predefined pairs of bodies each step (using the geometry from the URDF and Simscape's collision detection capabilities). If any distance falls below the safe threshold d_{safe} (indicating a potential collision or self-collision is imminent), the system triggers a collision event (see Figure 7) [38]. In our simulation, we set, e.g., $d_{safe} = 5$ cm as the minimum clearance. When triggered, the following occurs: (1) a warning flag is raised and logged (for analysis), (2) a done signal is sent to terminate the episode immediately, and (3) all joint torque commands from the agent are overridden to zero to stop the robot motion. This mechanism effectively acts as an emergency brake, preventing the agent from continuing once a collision is about to happen. The collision monitor was tested by deliberately commanding the arm toward the base. It reliably detected the violation of the safety distance and stopped the system (see Figure 8). During training of the RL agents, collisions occurred in early exploratory episodes for some agents, but the monitor caught them and ended those episodes with heavy penalties. As training progressed, all agents learned to avoid collisions altogether, indeed, in the final evaluation all agents achieved episode without any collision ($K5 = 0$), confirming the efficacy of this safety measure.

Joint Limit Enforcement: The robot arm has joint angle limits (e.g., each revolute joint might be limited to ±180° or a smaller range based on mechanical constraints). Violating joint limits could damage a real robot and often indicates an unstable controller. We enforce joint limits at two levels. First, in the Simscape model, we include mechanical joint limits with penalty forces (a stiff barrier that resists further motion beyond the limit). Second, we add an Assertion block that monitors each joint angle. If any joint exceeds its predefined limit, the assertion triggers a violation event. Similarly to the collision case, we handle a joint limit violation by: terminating the episode, applying a large negative reward, and zeroing out all torques immediately. In fact, the Simulink model is configured such that when the assertion triggers, it directly sets the torque inputs τ to zero in that same time step as a protective measure. This ensures the robot doesn't push further into the limit or oscillate. In our results, we observed very few limit violations once agents were trained. Among all algorithms, PPO and TRPO had a handful of episodes grazing the limits (K6 values of a few percent),

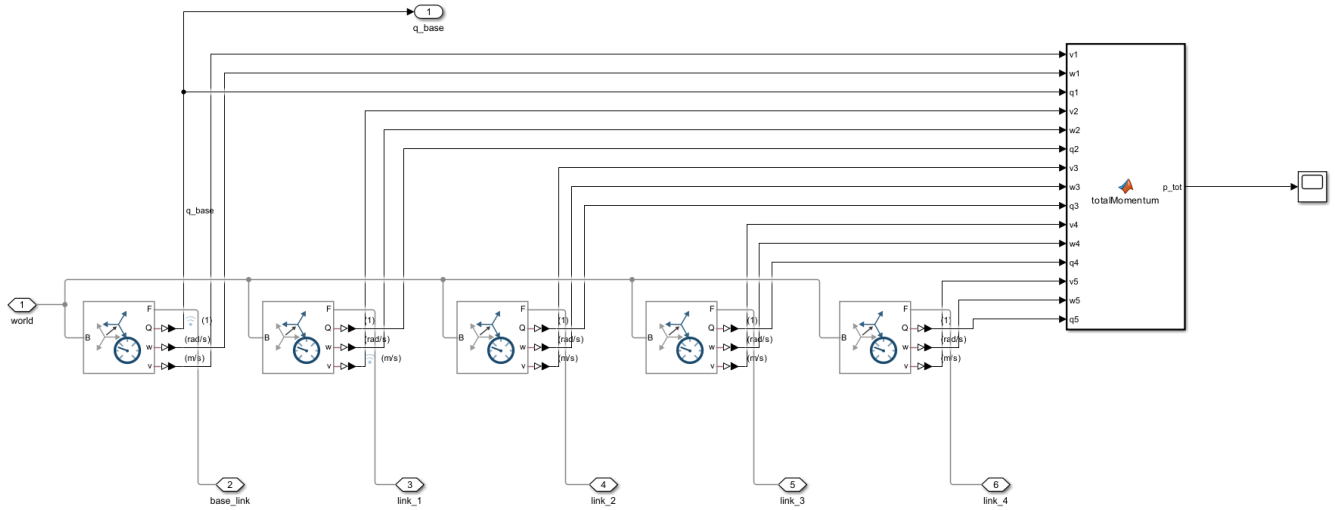


FIGURE 5: Overview of the Impuls Monitor Subsystem (Momentum Observer) in the Simulink Model `SpaceRobot.slx` This figure shows the computation of the total linear and angular momentum of the free-floating space robot. The velocities and orientations of the base and all manipulator links are measured and aggregated within a MATLAB function to obtain the system momentum. This monitor is used to analyze momentum conservation and quantify base–manipulator dynamic coupling effects.

whereas TD3, SAC, PG, and DDPG essentially never hit the limits ($K6 \approx 0$). Notably, PPO's superior tracking performance came at the cost of occasionally using the full joint range (K6 = 0.0505, i.e. ~5% of episodes, had minor limit violations). Thanks to the joint limit enforcement, these events did not lead to instability: the agent's torques were cut off momentarily when a limit was reached, preventing any damaging behavior, and the episode was marked as failed. Over time, the agent learned to avoid this scenario as well, especially in the optimized PPO version (where K6 dropped to 0).

Formal Verification of Torque Constraints: In addition to enforcing torque saturation during simulation, we performed an offline formal verification to prove that the trained policy could not command out-of-bounds torques. Using MATLAB's Simulink Design Verifier (SLDV) in property-proving mode, we analyzed a simplified closed-loop model of the agent controlling the robot's joints. The property to prove was that for all possible agent inputs (observations), each output torque τ_i remains within $[-\tau_{\max}, +\tau_{\max}]$ (see Figure 9). The verification succeeded: SLDV was able to prove that no counterexample exists and the torque limits are always respected by the policy network. Figure 10 shows the SLDV result confirming the property (no violations found) [22]. This formal assurance is important because it considers all possible states, including ones not seen during simulation, and guarantees the policy cannot output a dangerous torque. It's worth noting that directly running formal analysis on the full Simscape model was not feasible, the nonlinear dynamics are too complex for the automated theorem proving. To circumvent this, we extracted the relevant part of the model (the control law and saturation logic) into a separate simplified model (Verify_tau.slx) where the plant was replaced by a worst-case abstraction (essentially, we just needed to check the policy output block). This exemplifies how formal methods can complement simulation-based learning: by verifying critical properties on an abstract model of the agent, we gain additional confidence in safety beyond empirical testing.

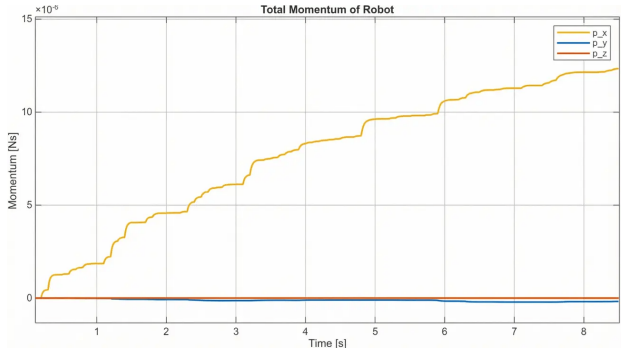


FIGURE 6: Dynamics of the total momentum of the robot.

Limitations of Formal Methods: While the above components significantly enhance safety, we observe that they cover a limited set of properties. They ensure zero collisions, no joint position overshot, and bounded torques, and they monitor physical consistency (e.g., momentum). However, other aspects, such as guaranteeing task completion within a time or stability margins, were not formally verified. A complete formal specification of the entire mission (for instance, something like temporal logic specifications of "the robot shall accomplish X without Y happening") would be far more complex. Our approach selectively targets the most critical safety concerns. In practice, this proved sufficient to train successful policies without a single catastrophic fail-

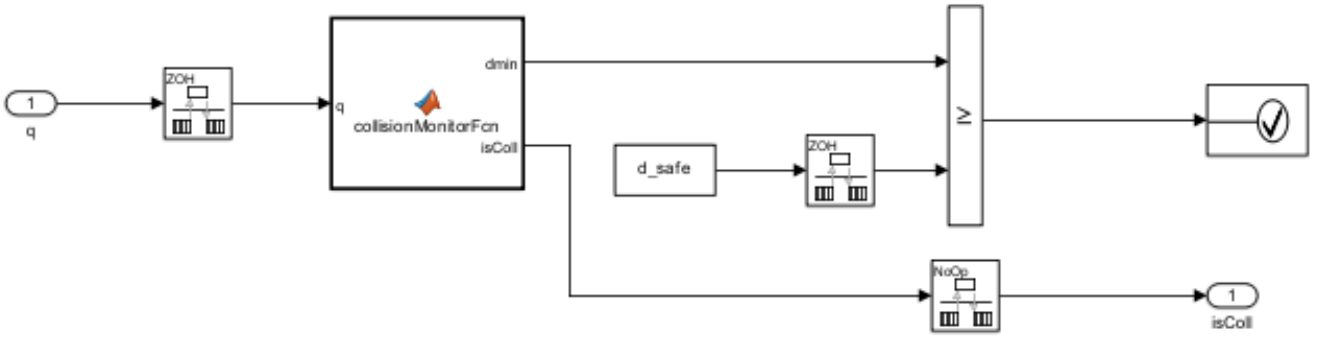
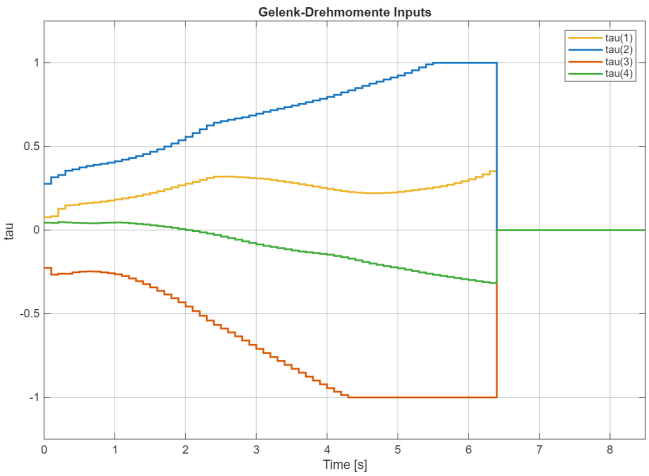


FIGURE 7: Overview of the Collision Monitor Subsystem in the Simulink Model `SpaceRobot.slx`
This figure illustrates the collision monitoring logic used to ensure safe robot operation during training and evaluation. Joint and link configurations are processed to compute the minimum distance to obstacles, which is compared against a predefined safety threshold. A collision flag is generated and used for episode termination or safety-related penalties.

(a) Automatically triggered collision warning



(b) Scope of torque inputs in the event of a collision

FIGURE 8: Representation of collision detection and the associated torque curves.

ure: episodes that would have been catastrophic were safely stopped by the monitors (and heavily penalized to teach the agent). As a result, by the end of training, the learned agent's behavior was entirely meaningful within the safety envelope enforced by the formal constraints [24].

In summary, integrating these formal safety checks into the learning process was vital. It not only protected the simulation (and a potential future real system) from damage during the exploratory phase of learning, but it also shaped the agent's behavior by penalizing unsafe actions. The agent thus learns with a form of built-in "safety shield." This approach demonstrates a viable path to apply deep RL in safety-critical domains like space robotics, where pure trial-and-error without safeguards would be unacceptable. We next analyze the performance outcomes of the various RL agents under these conditions.

VI. COMPARISON OF CONTINUOUS RL AGENTS

A central question of this study is: Which continuous-action RL algorithm is most effective for controlling a free-floating space robot? To answer this, we evaluated six RL algorithms from three major families on the same task and environment [21], [29]. The algorithms compared are:

- Policy Gradient (PG): the classic REINFORCE algorithm (Monte Carlo policy gradient), included as a baseline for direct policy optimization without advanced variance reduction [20], [21].
- Proximal Policy Optimization (PPO): a modern policy-gradient method that uses clipped probability ratio updates to ensure stable and conservative policy improvements. PPO is known for its balance of stability and performance [19].
- Trust Region Policy Optimization (TRPO): a policy-gradient method that enforces a trust-region constraint (via Kullback–Leibler divergence) on policy updates. TRPO is expected to perform similarly to PPO in terms

of stability, albeit with higher computational cost due to solving a constrained optimization each update [16].

- Deep Deterministic Policy Gradient (DDPG): an off-policy actor-critic algorithm with a deterministic policy and target networks. DDPG can excel in continuous control by learning a Q-function and a policy simultaneously, but it is known to be sensitive to hyperparameters and can be unstable [42].
- Twin Delayed DDPG (TD3): an improved variant of DDPG that addresses function approximation issues by using two critic networks to mitigate overestimation bias and by delaying policy updates. TD3 generally offers more stable learning than standard DDPG [17].
- Soft Actor-Critic (SAC): an off-policy actor-critic algorithm that learns a stochastic policy and includes an entropy term in the reward to encourage exploration. SAC aims to achieve both high reward and high entropy, making the agent's behavior robust and exploratory. It often achieves very good performance but can be computationally heavier due to sampling and entropy calculations [18].

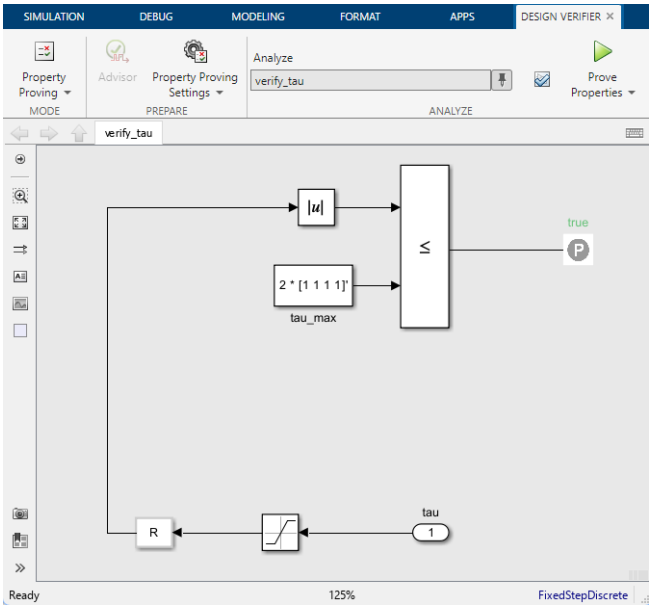


FIGURE 9: Verification model with Simulink Design Verifier `Verify_tau.slx`
This figure depicts the Simulink property model used to formally verify the joint torque constraints. The absolute joint torques are compared against predefined maximum limits using a relational operator. The property is proven to hold for all reachable system states, ensuring bounded control actions.

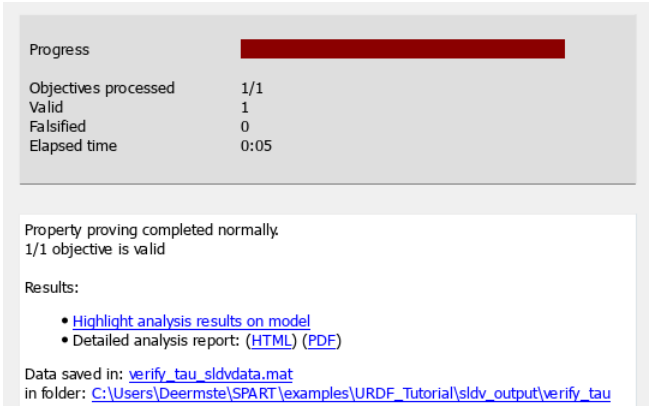


FIGURE 10: Result of the Simulink Design Verifier
This figure shows the result of a formal property verification performed on the torque constraint model. The verification confirms that the specified safety property is satisfied for all analyzed execution paths. This analysis provides formal guarantees on control torque limits beyond empirical simulation-based validation.

TABLE 3: Classification of the evaluated reinforcement learning algorithms.

Algorithm	Policy Type
Policy Gradient (PG)	On-policy
Proximal Policy Optimization (PPO)	On-policy
Trust Region Policy Optimization (TRPO)	On-policy
Deep Deterministic Policy Gradient (DDPG)	Off-policy
Twin Delayed DDPG (TD3)	Off-policy
Soft Actor-Critic (SAC)	Off-policy

These algorithms cover a spectrum from on-policy to off-policy, from deterministic to stochastic, and from simpler to more complex update rules. Based on prior knowledge and the nature of our problem, we hypothesized that algorithms with more robust, stable policy updates (like PPO and TRPO) would likely outperform others in this highly coupled, continuous control scenario. Off-policy methods (DDPG, TD3, SAC) might reach good performance as well, but could suffer from tuning sensitivity and stability issues given the complex dynamics and safety constraints. Indeed, DDPG and basic PG were expected to be the least reliable due to known tendencies for instability without additional regularization.

On-policy methods (PG, PPO, TRPO) update their policies using only data from the current policy, leading to stable but less sample-efficient learning. Off-policy methods (DDPG, TD3, SAC) reuse experience from a replay buffer, improving sample efficiency at the cost of higher sensitivity to hyperparameters [21], [29]. For free-floating space robot control, where strong nonlinear coupling and safety constraints demand reliable convergence, on-policy methods are expected to be advantageous.

Training Setup: All experiments were conducted on a workstation equipped with an AMD Ryzen 7 7700 processor and 32 GB of RAM, using MATLAB R2025b with the Reinforcement Learning Toolbox and Simscape Multibody. We used the same network architecture (initially two hidden layers of 128 units each for policy and for value function, where applicable) and same reward structure for all. Hyperparameters (learning rate, discount factor $\gamma = 0.99$, etc.) were initially set to default reasonable values and kept consistent across agents as much as possible. All agents were trained using a learning rate of $\alpha = 1 \times 10^{-3}$ for both

actor and critic networks, unless stated otherwise. Hence, any performance differences can be attributed primarily to the algorithmic differences, not to unequal tuning. Later we will tune PPO further, but for the main comparison, PPO uses a default configuration as well. Each agent's training run of 1000 episodes yielded a final policy which we then evaluated.

Performance Metrics: We evaluated the agents on the KPIs described earlier (tracking error, base stability, etc.) by running 30 test episodes for each. Table 4 summarizes a subset of these metrics, highlighting the differences between agents.

The results in Table 4 reveal clear patterns across the six agents. Regarding training stability, TRPO and PPO exhibited the most stable learning (T1 values of 0.61 and 1.78, respectively), whereas DDPG showed highly erratic behavior (T1 = 64.5). PPO was also the fastest to train (T3 $\approx$ 8 min), while off-policy methods required substantially longer (TD3 $\approx$ 38 min, SAC $\approx$ 35 min).

In terms of trajectory tracking, PPO achieved the lowest mean squared error (K2 = 0.0257), significantly outperforming the runner-up TRPO (0.0680). DDPG achieved the lowest peak error (K3 = 0.4758) in its best runs but was highly inconsistent. PG, SAC, and TD3 all exhibited substantially larger tracking errors, with PG's mean error roughly six times that of PPO.

For base motion minimization, PPO and TRPO both kept the base orientation error small (K4 of 0.0667 and 0.0694), while DDPG allowed the base to drift considerably (K4 = 0.2682). Interestingly, TD3 achieved very low base angular velocity (K8 $\approx$ 0.02 rad/s) by adopting an overly conservative strategy that sacrificed tracking accuracy. All agents successfully avoided collisions (K5 = 0). PPO and TRPO occasionally approached joint limits (K6 of 5.1% and 2.1%, respectively), indicating more aggressive use of the workspace for better tracking.

PPO also produced the smoothest control signals (K7 = 0.6812), while TD3 and DDPG generated the jerkiest outputs ($\approx$ 0.8–1.0). Energy consumption (K9) was moderate for PPO ($\approx$ 6.33) and lowest for TRPO ($\approx$ 3.76); the very low values for DDPG and TD3 ($<$ 1.0) reflect insufficient actuation rather than efficiency. Qualitatively, PPO produced the most competent behavior: the arm moved accurately along the circle with minimal base rotation. TRPO performed similarly but with slightly slower response. SAC tended to oscillate around the path, TD3 adopted an overly conservative strategy sacrificing tracking accuracy, DDPG was inconsistent across runs (explaining its high T1), and PG never achieved reliable tracking within 1000 episodes.

Overall Winner, PPO: Aggregating all the criteria, PPO stands out as the best-performing agent. As summarized succinctly in the report:

- PPO offers the best combination of training stability (low T1/T2), training speed (short T3), tracking accuracy (low K2/K3), and smoothness (low K7), with a moderate joint limit violation rate.
- TRPO is a close second, described as similarly stable and precise but requiring more computation time and still incurring slight limit violations.
- The off-policy methods (TD3, SAC) did not achieve better performance and were significantly more computation-intensive, with higher tracking errors.
- Finally, PG and DDPG were less robust both in training and in achieved performance, showing the largest fluctuations and poorest metrics.

These findings validate our hypothesis that on-policy methods with robust update rules are best suited for this highly coupled, safety-critical control problem. Based on these results, we selected PPO for further optimization.

Based on these results, we selected PPO as the reference agent for further development and optimization. The next section discusses how we improved PPO's performance even further through targeted optimizations.

VII. OPTIMIZATION OF THE PPO AGENT

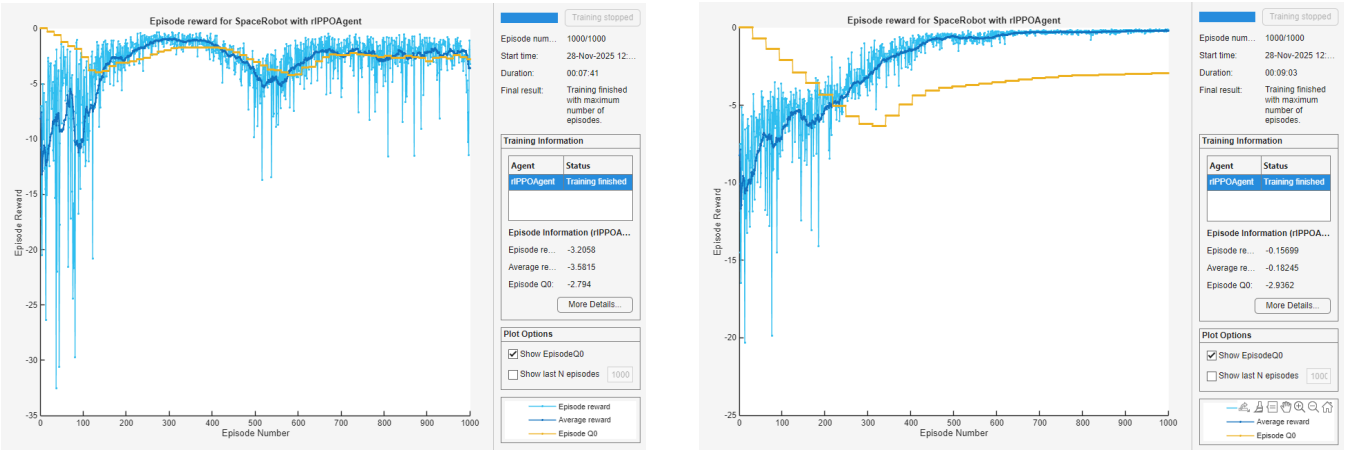
Having identified PPO as the most promising algorithm, we focused on enhancing its performance on path tracking task of the space robot. The PPO agent we compared above was a "default" PPO configuration (as provided by MATLAB's RL Toolbox example) with minimal tuning [41]. While it already outperformed other agents, we hypothesized there was room to improve in terms of faster convergence, higher precision, and eliminating the remaining constraint violations. We therefore undertook a systematic optimization of PPO, involving both network architecture modifications and hyperparameter tuning.

Network Architecture Improvements: By default, the PPO agent used two hidden layers of 128 neurons each (with ReLU activations) for both the policy and value function networks. We experimented with deeper and wider networks, as well as alternative activation functions, to better capture the dynamics of the problem. Our optimized architecture ultimately used 3 hidden layers for the policy network with layer sizes [256, 256, 128] and 2 hidden layers for the value network with sizes [256, 128]. We found that the additional depth and capacity in the policy network helped the agent learn the intricate relationship between arm motions and base reactions (especially given the 23-dimensional state input) [41]. We also introduced layer normalization in the first layer to help stabilize learning, given the different scales of input features (joint angles vs. orientation quaternions, etc.). The activation function remained ReLU, which was sufficient. These architecture changes were implemented by explicitly defining the neural network layers instead of using the auto-generated default network. In summary, the optimized policy network was larger and potentially more expressive, which we believe allowed PPO to find a control strategy which is better tuned.

Hyperparameter Tuning: We systematically varied key PPO hyperparameters and evaluated their impact on training curves (learning speed and stability):

TABLE 4: Selected performance metrics for each RL agent (best values in bold). PPO excels in most categories, achieving the lowest tracking errors (K2, K3), best base stability (K4), and smoothest control (K7), while maintaining short training time. TRPO is a close second in many metrics but requires more training time. Off-policy methods (TD3, SAC, DDPG) show larger errors or instabilities (e.g., high T1 for DDPG) despite some strengths (DDPG's low max error K3). KPI data from evaluation of 30 episodes per agent.

KPI (unit)	TRPO	TD3	SAC	PPO	PG	DDPG
K2: Mean Squared EE position error	0.0680	0.1914	0.1908	**0.0257**	0.1638	0.0937
K3: Max EE error	0.6783	0.5341	0.7766	0.4833	1.0654	**0.4758**
K4: Mean base orientation error	0.0694	0.0868	0.1687	**0.0667**	0.1302	0.2682
K6: Joint limit violation rate	0.0214 (2.1%)	0	0	0.0505 (5.1%)	0	0
K7: Torque smoothness (norm. jerk)	0.7498	0.9981	0.7536	**0.6812**	0.7125	0.8118
T1: Reward std. (late training)	**0.6144**	8.5168	9.6907	1.7756	13.1970	64.4764
T3: Training time (1000 eps)	∼13 min	∼38 min	∼35 min	**∼8 min**	∼8 min	∼24 min



(a) PPO (Default)

(b) PPO (Optimized)

FIGURE 11: Comparison of episodic rewards for default and optimized PPO agents. (a) Episode reward evolution during training using the default PPO hyperparameters. (b) Episode reward evolution using the optimized PPO configuration. The optimized configuration exhibits faster convergence and higher average episode rewards, indicating improved learning efficiency and policy performance.

- The learning rate was tuned: the default was 1e-3. We found that a slightly lower learning rate (e.g. 5e-4) improved stability (preventing overshooting in policy updates) without slowing convergence.
- The mini-batch size for stochastic gradient descent updates was increased. By default, PPO might use a batch of 64 from each trajectory. We increased this to 256 to provide more stable gradient estimates per update, which helped reduce the variance in training.
- The clip ratio (ϵ) for PPO's surrogate objective was adjusted. The default clip value 0.2 worked, but we tried slightly tighter (0.15) and looser (0.3) clips [19]. A tighter clip ($\epsilon = 0.1$–0.15) made updates more conservative. This reduced occasional policy overshoots at the cost of slower improvement. We settled on $\epsilon \approx 0.2$ as it was already near-optimal, but this process confirmed that too high a clip would harm stability. The clip factor ϵ in PPO limits the magnitude of policy updates by constraining the probability ratio between the new and the old policy. This prevents excessively large policy updates that could destabilize training, particularly in highly nonlinear control tasks. In safety-critical scenarios such as free-floating space robot control, clipping contributes significantly to stable and reliable convergence [19], [41].
- Entropy regularization weight was tuned. PPO can include an entropy term to encourage exploration. We increased the entropy coefficient modestly (from 0 to 0.01) to avoid premature convergence to a suboptimal deterministic policy. This indeed improved the exploration in early training, helping the agent discover better strategies (like moving joints in opposite directions to cancel base momentum).
- Discount factor (γ) and GAE (Generalized Advantage Estimation) parameter (λ) were left at standard values ($\gamma = 0.99$, $\lambda = 0.95$) after confirming that they already provided a good bias-variance trade-off for advantage estimation

- We also adjusted the target KL-divergence for PPO's stopping criteria (if used) to ensure the policy updates didn't diverge. In practice, we monitored the KL and it remained within acceptable range with our chosen settings.

Through a grid search over these parameters (one at a time and some combined), we converged on a set that provided a more stable and higher-performing PPO agent. Notably, the optimized agent is tailored to this specific task and reward function, for example, the entropy bonus was chosen to the point where it encourages trying different arm motions, but not so high that the agent keeps thrashing the arm aimlessly.

A. CIRCULAR TRAJECTORY

Training Outcome: The effects of these optimizations were evident in the training learning curves (see Figure 11). The default PPO agent's learning curve initially rose, but then plateaued at a fairly negative reward value (~-2 per episode) and exhibited high variance even after many episodes. This plateau suggested it reached a suboptimal policy and struggled to improve further. In contrast, the optimized PPO agent's learning curve kept rising steadily and converged to a much higher average reward (less negative, closer to zero) with significantly reduced variance. Table 5 reports the KPI averages over $N_{eval} = 30$ evaluation episodes for PPO with the default configuration and the optimized configuration. Specifically, where the default PPO leveled off around -2.0 cumulative reward, the optimized PPO approached -0.4 on average, a dramatic improvement. The variability (shaded region in the reward plot) shrank, indicating the agent's behavior became consistently optimal across training episodes. Quantitatively, the mean episodic reward K1 improved from -2.23 (default PPO) to -0.43 (optimized PPO). This roughly fivefold increase in reward corresponds to the agent accomplishing much more of the task (since zero would be perfect tracking with no penalties). The lower variance also hints that the agent not only improved its average performance but did so reliably without frequent failures.

Beyond the reward improvement, the optimizations translate directly to task performance. Figure 12 shows the end-effector path in the XY-plane: the default PPO deviates significantly from the reference circle and exhibits jagged oscillations, whereas the optimized PPO traces the reference almost exactly. The mean squared tracking error K2 dropped from 0.0257 to 0.0083 (a 67.7% reduction), and the maximum error K3 decreased from 0.4833 to 0.27.

Base stabilization improved markedly (Figure 13): the mean orientation error K4 fell from 0.0667 to 0.022 (factor ≈ 3), and the mean angular velocity K8 was halved from 0.095 to 0.050 rad/s. The optimized agent learned arm motions that effectively cancel momentum transfer to the base.

The optimized PPO also eliminated all safety violations (K5 = K6 = 0 over 30 episodes, compared to 5.1% joint-limit violations for the default), improved torque smoothness (K7: 0.6812→0.351), and reduced energy consumption by 16% (K9: 6.325→5.314). Notably, better tracking, base stability, safety, and energy efficiency were all achieved simultaneously, indicating a fundamentally more efficient control strategy.

The optimized PPO agent fulfills all task objectives simultaneously: precise trajectory tracking, minimal base disturbance, zero safety violations, smooth actuation, and reduced energy consumption. This validates the approach of systematic hyperparameter tuning and network tailoring for safety-critical space robot control.

TABLE 5: Performance of PPO agent before and after optimization. The optimized PPO shows substantial improvements in all aspects (higher rewards, lower errors, zero safety violations, smoother and more efficient control).

Metric (KPI)	PPO (Default)	PPO (Optimized)
Mean Episodic Reward (K1)	-2.23	$\mathbf{-0.43}$
Mean Squared EE Tracking Error (K2)	0.0257	**0.0083**
Max EE Tracking Error (K3)	0.4833	**0.27**
Mean Base Orientation Error (K4)	0.0667	**0.022**
Collision Rate (K5)	0	0
Joint Limit Violation Rate (K6)	0.0505 (5.1%)	**0**
Torque Smoothness (K7, jerk)	0.6812	**0.351**
Energy Consumption (K9)	6.325	**5.314**

B. PIECEWISE-LINEAR TRAJECTORY

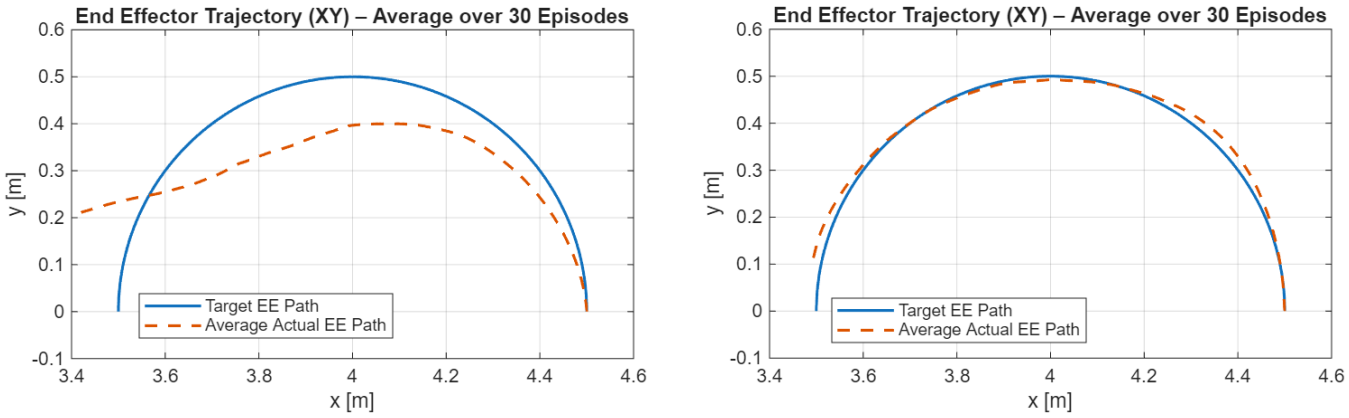
While the previous evaluations focused on tracking a smooth circular end-effector (EE) reference, we additionally tested how well the controller generalizes to linear trajectories. This is relevant for practical on-orbit manipulation tasks, where approach and retreat motions are frequently planned as straight-line segments rather than continuous curves.

Trajectory definition: the linear reference is constructed from the same circle parameters (center and radius) used for the circular benchmark, but the EE is commanded to move along two straight segments connecting three points on that circle: $P_0 = [c_x + r,\ c_y,\ c_z]$, $P_1 = [c_x,\ c_y + r,\ c_z]$, and $P_2 = [c_x - r,\ c_y,\ c_z]$. Using a period T and a switching time $t_1 = T/2$, the piecewise-linear reference position $p_{\mathrm{ref}}(t)$ is

$$p_{\mathrm{ref}}(t) = \begin{cases} P_0 + \frac{t}{t_1}\left(P_1 - P_0\right), & 0 \le t \le t_1, \\ P_1 + \frac{t-t_1}{T-t_1}\left(P_2 - P_1\right), & t_1 < t \le T. \end{cases} \tag{5}$$

The reference is sampled at $T_s = 0.01\,\mathrm{s}$, while the PPO policy is updated at $T_{s,\mathrm{agent}} = 0.1\,\mathrm{s}$ (same as in the other experiments).

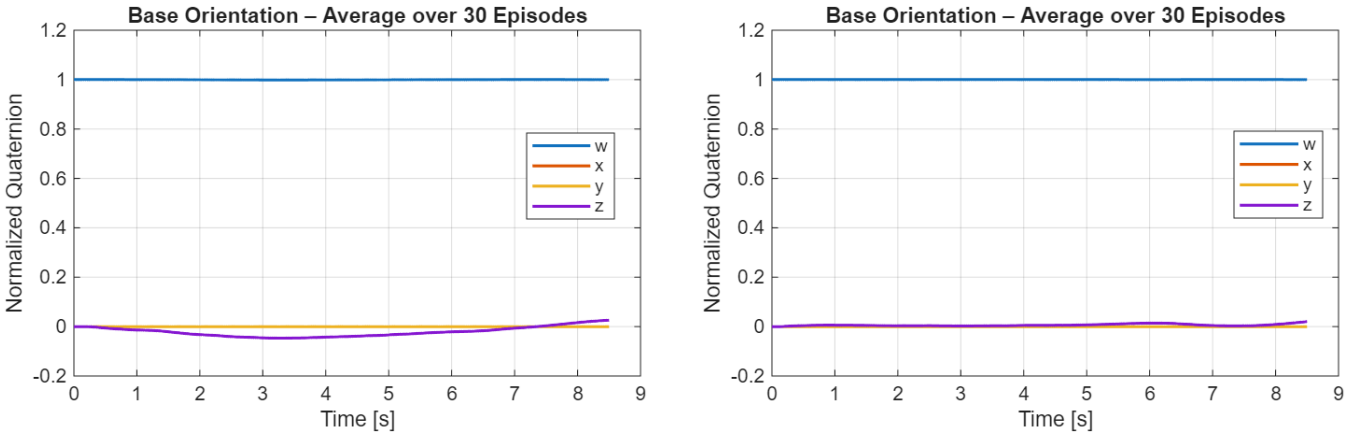
Results: Table 6 reports the KPI averages over $N_{eval} = 30$ evaluation episodes for PPO with the default configuration and the optimized configuration, now tested on the piecewise-linear reference. The optimized PPO improves overall task success (K1 increases from -0.4565 to -0.2977), reduces base attitude disturbance (K4 decreases by $\approx 44\%$) (see Figure 15), lowers residual base angular velocity (K8 decreases by $\approx 15\%$), and significantly reduces energy consumption (K9 decreases by $\approx 35\%$). The mean and peak end-effector tracking errors (K2 and K3) remain



(a) PPO (Default)

(b) PPO (Optimized)

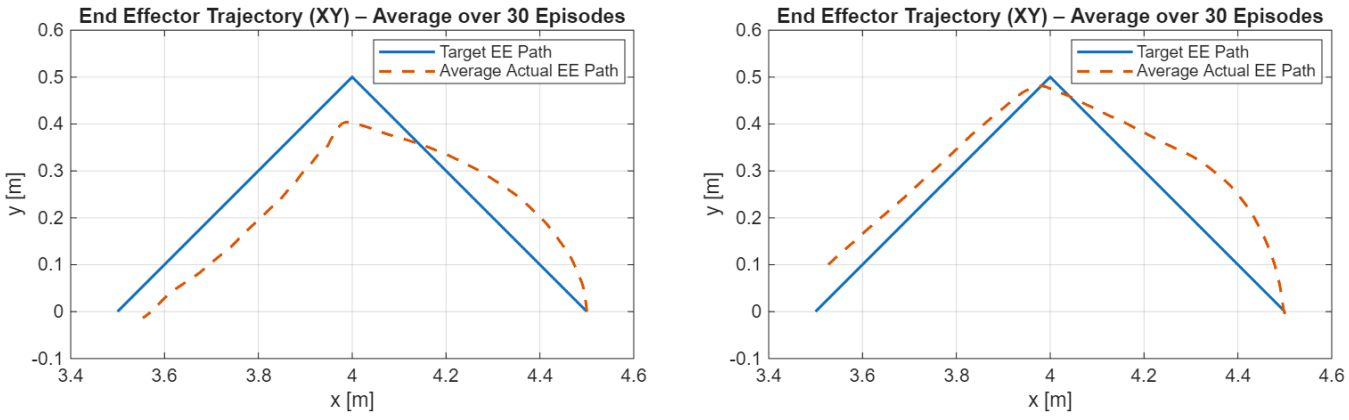
FIGURE 12: Target/actual comparison of the end-effector trajectory in the XY plane.



(a) PPO (Default)

(b) PPO (Optimized)

FIGURE 13: Course of base orientation (quaternion components) during a test episode.

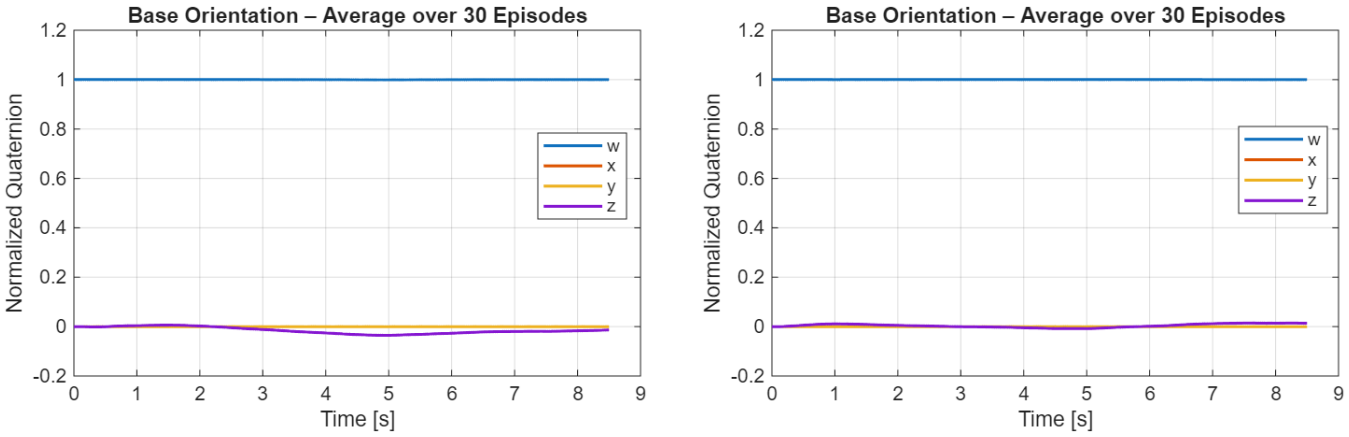


(a) PPO (Default)

(b) PPO (Optimized)

FIGURE 14: Target/actual comparison of the ramp end-effector trajectory in the XY plane.

almost unchanged, with a slight improvement in the mean error (see Figure 14). However, the control smoothness metric (K7) becomes worse than the default PPO. This suggests that the hyperparameter optimization (highly beneficial for circular tracking) does not fully transfer to piecewise-linear motion profiles. A straightforward mitigation is to include a mixture of reference types (circular, linear segments, and hybrid trajectories) during training (or during hyperparameter



(a) PPO (Default)

(b) PPO (Optimized)

FIGURE 15: Course of base orientation (quaternion components) during a test episode with ramp trajectory.

search) to explicitly optimize for trajectory generalization.

Both the circular and piecewise-linear evaluations validate the approach of systematic hyperparameter tuning and network tailoring for safety-critical space robot control, while also revealing that trajectory-specific optimization may be needed for optimal generalization.

In the next section, we will discuss the broader implications of these results, limitations, and future avenues to bring this work closer to deployment on physical space robot hardware.

VIII. DISCUSSION AND CONCLUSION

This work has presented a comprehensive study on applying continuous deep reinforcement learning to the control of a free-floating space robotic arm, with an emphasis on safety and performance. The results demonstrate that an RL-based controller can successfully handle the challenging manipulation task in a space environment, achieving high trajectory tracking accuracy while keeping the spacecraft base disturbance to a minimum. The learned agent (especially the optimized PPO policy) was able to coordinate the arm motions in such a way that the end-effector closely followed the desired path and the base's unintended motions were small and within acceptable bounds, a key requirement for on-orbit operations. The training process exhibited the typical behavior of RL: an initial exploration phase where performance was poor (and the agent occasionally triggered safety mechanisms), followed by steady improvement as the agent learned from experience, and finally a convergence to a stable policy indicated by decreasing variance in returns. The decreasing variability of the episodic reward and error metrics suggests that the agent's behavior became reliably repeatable and less stochastic as it honed in on an optimal strategy [1], [21].

TABLE 6: Generalization test on a piecewise-linear end-effector trajectory (two-segment ramp). Values are averaged over $N_{eval} = 30$ episodes. Bold indicates the better value according to the KPI definition (K1 higher is better; K2–K9 lower is better).

Metric (KPI)	PPO (Default)	PPO (Optimized)
Mean Episodic Reward (K1)	-0.4565	$\mathbf{-0.2977}$
Mean Squared EE Position Error (K2)	0.0078	**0.0070**
Max EE Position Error (K3)	0.2037	**0.2036**
Mean Base Orientation Error (K4)	0.0412	**0.0231**
Collision Rate (K5)	0	0
Joint Limit Violation Rate (K6)	0	0
Control Smoothness (K7, jerk)	**0.5520**	0.6839
Mean Base Angular Velocity (K8)	0.0399	**0.0339**
Energy Consumption (K9)	7.7144	**5.0341**

One notable achievement is that the agent can generalize to different initial conditions (within the distribution it was trained on). We tested the final PPO agent on various starting arm configurations and positions along the trajectory. It consistently managed to guide the end-effector to the circle and then track it, as long as those initial states were within the training range. This indicates that the policy is not simply memorizing a sequence of actions, but truly learning a feedback control law that is robust to moderate variations in the initial state. However, this generalization only holds within the space of states it experienced. Drastic changes (like a completely different trajectory or a very different initial base orientation) were not tested and would require either retraining or additional training with such scenarios (perhaps via domain randomization [43]).

Practical Implications: The success of PPO (and TRPO) in this domain suggests that policy gradient methods are well-suited for complex, continuous control tasks with coupled dynamics and safety constraints. In particular, PPO's ability to handle continuous action spaces and its stable update rule (clipped surrogate objective) made it robust in the face of the highly nonlinear dynamics of the free-floating robot. The integrated safety components (collision avoidance, etc.) effectively acted as a form of "shielding" during training, they prevented the agent from exploring disastrously unsafe

actions by cutting those episodes short [19], [23], [24]. This, combined with the penalty-driven learning, means the final policy inherently respects those constraints (since it learned quickly that violating them yields no reward). For real missions, this is very promising: it implies we can embed safety checks in the training simulator and reasonably expect the trained policy to abide by them when deployed (assuming the real system is within the simulator's fidelity) [23], [30].

Limitations and Future Work: Several limitations should be acknowledged. First, a sim-to-real gap remains: unmodeled effects such as flexible dynamics, sensor noise, actuator latency, and external disturbances (gravity gradients, magnetic torques) could degrade real-world performance [23], [43]. Second, the RL agent exhibits hyperparameter sensitivity—the tuned PPO configuration is task-specific and may require re-tuning for different trajectories or mission scenarios. Third, the state representation was manually designed; learned representations or recurrent architectures (LSTM) might improve performance under partial observability. Fourth, the safety monitoring covers targeted critical properties but does not constitute exhaustive formal verification of the full mission [22], [23]. Finally, no hardware testing was performed; real-time computation limits, communication delays, and physical interactions remain to be validated.

Future work should address these gaps through: (i) domain randomization over simulation parameters to improve robustness for sim-to-real transfer [43]; (ii) extension to more complex tasks such as dual-arm coordination, on-orbit assembly, or active debris removal; (iii) advanced RL architectures (recurrent policies, attention mechanisms) and safe RL algorithms with explicit constraint handling via Lagrange multipliers or barrier functions [23], [24], [30], [31]; and (iv) hardware-in-the-loop validation on robotic testbeds with simulated microgravity (e.g., air-bearing tables).

Conclusion: In conclusion, this research demonstrates a viable framework for applying deep reinforcement learning to complex space robotic systems. We showed that by combining high-fidelity modeling, multiple RL algorithm evaluations, and integrated formal safety checks, one can develop a control policy that achieves high precision and reliability in a challenging free-floating manipulation task. The comparative analysis revealed PPO as an effective solution, and further optimizations yielded a policy that meets stringent trajectory and safety requirements. Importantly, the use of formal methods alongside RL is a novel contribution, it addresses the typical black-box nature of learned controllers by adding verifiable guarantees for certain behaviors. This synergy of model-based and data-driven methods is very promising for space systems, where safety cannot be compromised.

The work also provides insights into the influence of algorithm choice and parameter tuning on learning performance for physical systems. Our findings reinforce the notion that there is no one- size-fits-all in RL, careful selection and tuning of the agent (and network structure) can significantly impact the outcome, even more so than we might expect from generic benchmarks. For practitioners, this means that investing time in benchmarking algorithms (as we did) can pay off in identifying the right approach for the problem at hand.

Finally, while challenges remain for real-world deployment, this study lays a foundation. It suggests that future space robots, tasked with servicing satellites, assembling structures, or handling space debris, could employ learning-based controllers that adapt to complex dynamics and uncertainties, all while operating safely within predefined limits. We envision an architecture where an RL policy drives the robot's motions, supervised by a layer of formal logic that monitors and vetoes any unacceptable actions (though, ideally, the policy has learned not to propose such actions in the first place). Such an approach could provide the flexibility and efficiency of learned behaviors with the trust and guarantee of model-based safety, a combination well-suited to the unforgiving environment of space [1], [24].

Ultimately, our work demonstrates a step in that direction, confirming that with the right algorithms and safeguards, "autonomous learning-based control" can be a powerful paradigm for future space robotic missions. The results encourage continued research at the intersection of robotics, reinforcement learning, and formal methods to further unlock the potential of intelligent space systems.

REFERENCES

[1] A. Flores-Abad, O. Ma, K. Pham, and S. Ulrich, "A review of space robotics technologies for on-orbit servicing," Prog. Aerosp. Sci., vol. 68, pp. 1–26, 2014, doi: 10.1016/j.paerosci.2014.03.002.

[2] K. Nanos and E. G. Papadopoulos, "On the dynamics and control of free-floating space manipulator systems in the presence of angular momentum," Front. Robot. AI, vol. 4, p. 26, 2017, doi: 10.3389/frobt.2017.00026.

[3] E. Papadopoulos, I. Kaliakatsos, and D. Psarros, "Minimum fuel techniques for a space robot simulator with a reaction wheel and PWM thrusters," in Proc. Eur. Control Conf. (ECC), 2007, pp. 3391–3398.

[4] E. Papadopoulos and A. Abu-Abed, "Design and motion planning for a zero-reaction manipulator," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 1994, pp. 1554–1559, doi: 10.1109/ROBOT.1994.351367.

[5] M. Mühlbauer, M. Chalon, M. Ulmer, and A. Albu-Schäffer, "Software for the SpaceDREAM robotic arm," in Proc. IEEE Aerosp. Conf., 2025, pp. 1–10.

[6] T. Rybus, "Robotic manipulators for in-orbit servicing and active debris removal: Review and comparison," Prog. Aerosp. Sci., vol. 151, p. 101055, 2024.

[7] H. Maurenbrecher et al., "RoboGrav—Towards force sensitive space manipulators," in Proc. I-SAIRAS, 2024.

[8] Z. Huang, G. Chen, Y. Shen, R. Wang, C. Liu, and L. Zhang, "An obstacle-avoidance motion planning method for redundant space robot via reinforcement learning," Actuators, vol. 12, no. 2, p. 69, 2023.

[9] A. Al Ali and Z. Zhu, "Reinforcement learning for path planning of free-floating space robotic manipulator with collision avoidance and observation noise," Front. Control Eng., vol. 5, p. 1394668, 2024.

[10] M. D'Ambrosio, L. Capra, A. Brandonisio, and M. Lavagna, "Failure management via deep reinforcement learning for robotic in-orbit servicing," in Proc. SPAICE—1st Joint ESA/IAA Conf. AI Space, 2024, pp. 335–339.

[11] A. Al Ali, J. Shi, and Z. Zhu, "Path planning of 6-DOF free-floating space robotic manipulators using reinforcement learning," Acta Astronaut., vol. 224, pp. 367–378, 2024.

[12] Y. Hu, D. Zhou, W. Yao, X. Shao, and G. Sun, "Deep reinforcement learning-based trajectory planning with continuous pose representation for 6-DoF free-floating space robot," Aerosp. Sci. Technol., p. 110540, 2025.

[13] O. Zhang, Z. Liu, X. Shao, W. Yao, L. Wu, and J. Liu, "Learning-based task space trajectory planning framework with pre-planning and post-

processing for uncertain free-floating space robots," IEEE Trans. Aerosp. Electron. Syst., early access, 2025.
[14] Y. Wei, X. Bai, and H. Lu, "Trajectory planning of free-floating space robot for non-cooperative tumbling target capture based on deep reinforcement learning," Robotica, vol. 43, pp. 2674–2692, 2025.
[15] Y. Wu, Z. Yu, C. Li, M. He, B. Hua, and Z. Chen, "Reinforcement learning in dual-arm trajectory planning for a free-floating space robot," Aerosp. Sci. Technol., vol. 98, p. 105657, 2020.
[16] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in Proc. Int. Conf. Mach. Learn. (ICML), 2015, pp. 1889–1897.
[17] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in Proc. Int. Conf. Mach. Learn. (ICML), 2018, pp. 1587–1596.
[18] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in Proc. Int. Conf. Mach. Learn. (ICML), 2018, pp. 1861–1870.
[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
[20] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," Mach. Learn., vol. 8, pp. 229–256, 1992.
[21] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
[22] MathWorks, "Simulink Design Verifier," The MathWorks, Inc., Natick, MA, USA, 2024. [Online]. Available: https://www.mathworks.com/help/sldv/
[23] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," J. Mach. Learn. Res., vol. 16, pp. 1437–1480, 2015.
[24] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in Proc. AAAI Conf. Artif. Intell., 2018, pp. 2669–2678.
[25] Open Robotics, "URDF (Unified Robot Description Format)," ROS 2 Documentation, 2024. [Online]. Available: https://docs.ros.org/en/rolling/Tutorials/Intermediate/URDF/URDF-Main.html
[26] MathWorks, "Import URDF models," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/sm/ug/urdf-import.html
[27] MathWorks, "smimport—Import a CAD, URDF, or Robotics System Toolbox model," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/sm/ref/smimport.html
[28] MathWorks, "Mechanism Configuration," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/sm/ref/mechanismconfiguration.html
[29] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," Int. J. Robot. Res., vol. 32, no. 11, pp. 1238–1274, 2013.
[30] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in Proc. Int. Conf. Mach. Learn. (ICML), 2017, pp. 22–31.
[31] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in Proc. Eur. Control Conf. (ECC), 2019, pp. 3420–3431.
[32] F. Leutert et al., "AI-enabled cyber–physical in-orbit factory—AI approaches based on digital twin technology for robotic small satellite production," Acta Astronaut., vol. 217, pp. 1–17, 2024.
[33] M. Mühlbauer et al., "Towards an in-orbit cubesat factory," in Proc. 76th Int. Astronaut. Congr. (IAC), 2025.
[34] O. Maqbool and J. Roßmann, "Systems aware scenario engineering for life cycle-spanning verification and validation of complex CPS," Authorea Preprints, 2025.
[35] J. Virgili-Llop, "SPART: Spacecraft Robotics Toolkit," 2017. [Online]. Available: https://github.com/NPS-SRL/SPART
[36] E. Ribeiro, R. Mendes, M. Terra, and V. Grassi, "Dynamic parameter identification of a 7-DoF lightweight robot manipulator using probabilistic differential optimization," in Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC), 2022, pp. 1917–1923.
[37] MathWorks, "6-DOF Joint (Simscape Multibody)," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/sm/ref/6dofjoint.html
[38] MathWorks, "Spatial Contact Force," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/sm/ref/spatialcontactforce.html
[39] MathWorks, "Solver Configuration," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/simscape/ref/solverconfiguration.html
[40] MathWorks, "Fixed-step size (fundamental sample time)," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/simulink/gui/fixedstepsizefundamentalsampletime.html
[41] MathWorks, "Reinforcement Learning Toolbox—rlPPOAgent," MATLAB & Simulink Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/reinforcement-learning/ref/rl.agent.rlppoagent.html
[42] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," in Proc. Int. Conf. Learn. Represent. (ICLR), 2016.
[43] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2017, pp. 23–30.

PATRICK S. ERMISCH Patrick S. Ermisch was born in Frankfurt am Main, Germany, on December 18, 2002. He received the Abitur degree and is currently pursuing the B.Eng. degree in mechatronics at Frankfurt University of Applied Sciences, Frankfurt, Germany, with an expected graduation in 2026. His major field of study is mechatronics.

He has gained practical experience through an internship at SEW-EURODRIVE, where he contributed to the further development of a mobile application for the manual control of logistics robots. In addition, he worked as a tutor for Physics I and Physics II and as a student research assistant, supporting research on particle simulation and the development of an application for visualization and simulation purposes. His research interests include reinforcement learning for robotics, space robotic systems, and simulation-based control methods.

Mr. Ermisch has been listed on the Dean's List of Frankfurt University of Applied Sciences multiple times in recognition of his academic performance.

SECOND B. AUTHOR was born in Greenwich Village, New York, NY, USA in 1977. He received the B.S. and M.S. degrees in aerospace engineering from the University of Virginia, Charlottesville, in 2001 and the Ph.D. degree in mechanical engineering from Drexel University, Philadelphia, PA, in 2008.

From 2001 to 2004, he was a Research Assistant with the Princeton Plasma Physics Laboratory. Since 2009, he has been an Assistant Professor with the Mechanical Engineering Department, Texas A&M University, College Station. He is the author of three books, more than 150 articles, and more than 70 inventions. His research interests include high-pressure and high-density nonthermal plasma discharge processes and applications, microscale plasma discharges, discharges in liquids, spectroscopic diagnostics, plasma propulsion, and innovation plasma applications. He is an Associate Editor of the journal Earth, Moon, Planets, and holds two patents.

Dr. Author was a recipient of the International Association of Geomagnetism and Aeronomy Young Scientist Award for Excellence in 2008, and the IEEE Electromagnetic Compatibility Society Best Symposium Paper Award in 2011.